

Syntax & Vocab Overview

Basic data types

A variable can hold many types of data. Some of the basic types of data that we have learned about so far in python are **integers**, **floats**, and **strings**. An **integer**, or **int**, is simply a whole number, and it can be negative as well as positive. A **float** (which stands for “floating point number”) is basically just a decimal. And a **string** is text data. One more data type that we haven’t really talked about is the **boolean** (pronounced boo-lee-un), which can either be **True** or **False**, and that’s it.

Examples:

- -354 is an **int**
- 43.7120 is a **float**
- "I have an apple in my pocket" is a **string**
- True is a **boolean**

When you type in a value directly like this, it is called a **literal**. The alternative is to store the value in a **variable**.

Assigning or setting a variable

A **variable** is a symbol (often a letter or a word joined by underscores) that holds a **value**.

Examples:

`x = 57` sets the **variable** `x` to have the value 57

`new_pitch = pitch + 10` sets the **variable** `new_pitch` to be whatever the value of the **variable** `pitch` is plus 10

Operators and doing math

You can do addition, subtraction, multiplication, and division in Python by using the **+**, **-**, *****, and **/** **operators**. An **operator** is defined as something that takes two values and puts them together in some way. You can also do exponents by using the double asterisk **operator** ******.

Examples:

`pitch + 3` is equal to whatever `pitch` was plus 3

`pitch - 3` is equal to whatever `pitch` was minus 3

`volume * 1.5` is equal to whatever `volume` was times 1.5

`1.5 * volume` is exactly the same. Operators work just like they do in math: it doesn’t matter which order you multiply or add in.

`s.tempo / 2` is equal to whatever the tempo of the session “s” is divided by 2

Usually, **operators** are combined with assigning or setting a variable. So you might write `s.tempo = 180 / x` to set the tempo of the session `s` to be 180 divided by the value of the variable `x`.

Try it! [[ANSWERS](#)]

1. Write a line of code that sets the variable `name` to be equal to your name. Which type of data did you use?
 2. Suppose you had a car that drove 120 miles in 3 hours. Write a line of code that...
 - A. Sets the variable `distance` equal to 120
 - B. Sets the variable `time` equal to 3
 - C. Calculates and sets the variable `rate` based on the distance formula (distance = rate * time). This will involve rearranging the distance formula to get “rate” in terms of distance and time.
-

Changing the value of a variable

You can add one to a **variable** by **assigning** it to be one more than it was. For example, you could write `pitch = pitch + 1` to **set** `pitch` to be one more than it currently is. However, python has a simpler, more convenient **syntax** for doing this kind of thing: the `+=`, `-=`, `*=`, and `/=` operators.

Examples:

`pitch += 1` adds one to the **variable** “pitch”

`volume *= 0.9` multiplies the volume by 0.9, making it 90% of its previous **value**

`length /= 2` sets the length to be half of what it was

Try it! [[ANSWERS](#)]

1. Write a line of code that takes the variable `length`, and makes it go up by a factor of 1.2 or 120%
2. Write a line of code that makes the variable `num_bananas` go down by a factor of 3
3. Write 3 line of code:
 - A. One that **sets** the **variable** `pitch` to 65
 - B. One that **sets** the **variable** `pitch_change_amount` to some value (say, 2)
 - C. One that adds `pitch_change_amount` to `pitch`, so that the new value of `pitch` is 67

Calling functions

A **function** is piece of code that does something. The syntax for **calling** a function looks like a lower-case word, or several lower-case words connected with underscores, followed by parentheses. Sometimes, extra information about what we want to do (called **arguments**) is put inside of the parentheses. Often a **function** also gives you information back (**returns** a **value**).

For example:

`eat_tacos(5)` might tell the computer to eat 5 tacos. It would *not* **return** anything.

`get_lyrics("King Kunta", "Kendrick Lamar")` might ask the computer to get the lyrics for the song “King Kunta”. This **function** takes two **arguments**, the name of the song, and the name of the artist. The computer expects them in that exact order. This function would **return** something: the lyrics to the song (as a **string**).

Try it! [[ANSWERS](#)]

Assuming that someone has created the functions you are calling:

1. Write a line of code that tells the computer to shut down in 20 second
2. Write a line of code that asks the computer how many people in the US have green eyes and a goatee.
3. Which one of these functions would have a **return value**?

Keyword Arguments

There are two types of arguments in Python: **positional arguments**, and **keyword arguments**.

- With **positional arguments**, the computer understands what they refer to because of the order. For example, in `scamp`, the `play_note` function always takes the pitch argument first, followed by the volume argument, and then the length argument.
- With **keyword arguments**, a keyword is given (followed by an equals sign) to tell the computer what the argument is referring to. For example, in `play_note(60, 1.0, 3.0, blocking=False)`, “`blocking=false`” is a keyword argument. Keyword arguments can happen in any order, but they must happen after all of the positional arguments. Often, keyword arguments are optional, whereas positional arguments are required.

For example:

Suppose that the function `eat_tacos` takes two **positional arguments**: the first argument is the number of of tacos, and the second argument is the type of taco. It can also take two optional **keyword arguments**: `radius`, which determines the size of the tortilla, in inches; and `onions`, which determines whether or not there are onions. We might call the function as follows:

- `eat_tacos(3, "carnitas")` would tell the computer to eat three carnitas tacos. In this case, the values for `radius` and `onions` would default to whatever values the programmers think are appropriate, so for this example likely a standard street taco size, with onions.
 - `eat_tacos(2, "pastor", radius=12, onions=False)` would tell the computer to eat 2 pastor tacos, made on tortillas that are 12 inches wide, with no onions. We could have instead written `eat_tacos(2, "pastor", onions=False, radius=12)`. However, we cannot write `eat_tacos(2, onions=False, "pastor", radius=12)` because the **keyword arguments** must happen after all of the **positional arguments**.
-

Try it! [\[ANSWERS\]](#)

Suppose someone has created a function with which you can ask the computer to sing a song. The function expects two **positional arguments**: the song title, and the voice type (for example “high male”, “low female”). It can also, optionally take a keyword argument, `repeats`, which can be used to sing the song multiple times. What would you write to ask the computer to:

1. Sing you a specific song (once) with a low male voice
 2. Sing the song 57 times in a row, with a high female voice
-

Objects and classes

Python, like other **object-oriented** programming languages, have both **classes** (or **types**) and **objects**. A **class** is a type of object from which you can create many **instances**, or specific examples. The syntax for a **class** is a capitalized word, or multiple capitalized words joined together without underscores. For instance `Session` and `ScampInstrument` are both classes.

You can figure out the **class** of an object by using the `type` function. For example, if we want to print the type of the variable `s` we could write `print(type(s))`

Creating an object

The way you create an object of a particular class is very similar to how you call a function: You put parentheses after the class name, and both **positional** and **keyword arguments**

can go inside those parentheses. The difference is the class is a capitalized word, or multiple capitalized words joined together without underscores.

For example:

- We can create a session object with the code `Session()`
- Sometimes, we might want to give more information. For instance, `Session(tempo=100)` uses the **keyword argument** `tempo` to create a Session that has a tempo of 100.
- `my_cat = Cat("Minnie", "tortoiseshell", legs=3.5)` creates a new Cat object named Minnie and stores it in the variable `my_cat`. The first **positional argument** is used to tell the computer that her name is “Minnie”. The second **positional argument** is used to tell the computer her fur color; in this case, tortoiseshell. Finally, the **keyword argument** `legs` is used to tell the computer that Minnie has only 3 and a half working legs (she has some issues with the fourth leg).
- Note: In this example, we are creating the Cat object directly. Sometimes, however, **functions** create objects for us, as part of what they do. For example in the line `piano = s.new_part("piano")` the function `new_part` actually creates and then **returns** a new instrument object, without us having to create one directly.

Try it! [ANSWERS]

1. Suppose the class `Human` takes one positional argument, which is the person’s name. It can also take several keyword arguments for `hair_color`, `eye_color`, and `personality`. Write a line of code to create an **object** of **class** `Human` that represents you or someone you know, and assign it to a variable.

Dot Notation

When we want to ask an object to do something, we use dot notation. For example, `piano.play_note(60, 0.7, 1.0)` asks the piano object (which is of **class** `ScampInstrument`) to play a note. The object to the *left* of the dot is asked to perform the action on the *right* of the dot.

We haven’t really discussed this, but in this context, a **function** is usually called called a **method**. A **method** is a **function** performed by an **object**.

Examples:

- `my_cat.scratch_post()` would ask my cat (an **object** of **class** `Cat`, stored in the **variable** `my_cat`) to scratch its scratching post.
- `car.accelerate_to(60)` would ask the car object to accelerate to 60 mph.

Objects can also have **variables** associated with them. For instance, we have seen that **Session objects** have a variable called `tempo`, which affects the speed and length of all notes

played back by instruments in that session. Again, to use or change these variables, we use dot notation:

Examples:

- `s.tempo = 120` sets the tempo of the Session object `s` to 120 beats per minute.
- `my_cat.lives -= 1` reduces the number of lives that `my_cat` has by one. (Probably it started with 9 by default when it was created.)
- `x = my_cat.legs - 2` would **set** the **variable** `x` to be equal to the number of legs that `my_cat` has minus 2. Based on the way we constructed `my_cat` above, this would make `x` equal to 1.5.

Again, we haven't really discussed this, but in this context, a **variable** is usually called called an **attribute**. An **attribute** is a **variable** attached to an **object**.

Try it! [[ANSWERS](#)]

After creating your `Human` object above, add a couple of lines of code that:

1. Ask the person to do 20 push-ups
2. Change the person's name to "sparkie"
3. Give the person a haircut, by reducing their `hair_length` attribute. (Hint: consider using one of the `+=`, `-=`, `*=`, or `/=` operators)

Solutions 1

1. I would write `name = "Marc"`. "Marc" is a **string**, since it contains text data.
2. A. `distance = 120`
 B. `time = 3`
 C. `rate = distance / time`. In this case, rate will end up being set to 40, but if you had put different values for distance or time, it would turn out differently.

[Back to instructions](#)

Solutions 2

1. `length *= 1.2`
2. `num_bananas /= 3`
3.
 - A. `pitch = 65`
 - B. `pitch_change_amount = 2`
 - C. `pitch += pitch_change_amount`

[Back to instructions](#)

Solutions 3

Your answers might vary slightly, but should be something like this:

1. `shut_down(20)`
2. `num_people("US", "green", "goatee")`. You might instead have something like `num_people_in_us("green", "goatee")`, which is just fine. The difference is that in this case it is baked into the function that it's looking only at the US, whereas in the first case, the function takes the country of interest as an **argument**
3. `num_people_in_us` would have a return value

[Back to instructions](#)

Solutions 4

Something like:

1. `sing_song("Never Gonna Give You Up", "low male")`
2. `sing_song("Never Gonna Give You Up", "high female", repeats=57)`

[Back to instructions](#)

Solutions 5

1. I might write something like this:

```
my_friend = Human("Stewart", hair_color="brown",  
                  eye_color="blue", personality="friendly")
```

[Back to instructions](#)

Solutions 6

Having created my friend Stewart with the line:

```
my_friend = Human("Stewart", hair_color="brown",  
                  eye_color="blue", personality="friendly")
```

...I could then write:

1. `my_friend.do_push_ups(20)`
2. `my_friend.name = "sparkie"`
3. `my_friend.hair_length /= 2`

[Back to instructions](#)